

M.EEC041 - Digital Systems Design

2024/2025

Laboratory exercise 1

October 2024

Objectives: practice with iverilog and GTKwave; modify a given design based on a set of Verilog modules and improve the testbenches for testing new functionalities.

1 Introduction

The complete project should be downloaded from Moodle (laboratory exercise 1 - lab1.zip) and installed in an appropriate directory created under your home directory. If you already downloaded the archive, please do it again because some of the files have been updated. The archive contains 3 directories that should be used with specific purposes (a similar structure will be used for future projects):

- ./doc documentation (this guide)
- ./src any source files. Keep the Verilog files under a subdirectory called verilog
- ./sim run simulations here

In the first introductory exercise (lab classes of 30/set and 1/oct) it was proposed to analyze the Verilog modules from the top-level (modules `updowncounter8b` and `four_bit_adder_sub.v`) to draw detailed block diagrams that represent the circuits, along all their hierarchy levels. This exercise will focus only on the up/down counter.

The Verilog module `updowncounter8b` (file `updowncounter8b.v`) implements a 8-bit up-down counter with a counting step defined as a 8-bit input. This design was built as a hierarchical structural model, using some other simple modules that implement a full-adder, a 4-bit adder, a D-type flip-flop and an 8-bit register. The testbench `updowncounter_tb.v` is used to verify the functionality of this module.

2 Implementation

In this exercise you should modify the model of the counter for including two main functionalities:

- a) implement a load function to allow loading a new state to the counter register
- b) add additional logic to the counter, based on a comparator, to set the maximum value the counter will count before restarting with zero.

The new module should have three additional inputs as in the module `updowncounter8b_loadmax.v` that implements the original counter including these additional inputs:

```
input      load,      // set to 1 to load the counter register with data
                        // at the input din_load[7:0]
input [7:0] din_load, // data input to load to the counter register
input [7:0] max_count // maximum counting value
```

2.1 The load function can be implemented with an 8-bit 2-1 multiplexer. In normal counting mode this selects to the register input the output of the adder/subtractor, as in the current counter; in load mode, when input load is 1, the register will be loaded with the `din_load[ ]` input data. For now ignore the input `max_count[ ]`.

- a) draw a block diagram illustrating the modification required to the original circuit;

- b) build a functional (or behavioral) module that implements an 8-bit 2-1 multiplexer using the Verilog **always @*** statement (refer to the examples presented in the theoretical class).
- c) Modify the module `updowncounter8b_loadmax.v` to implement the block diagram created in a), using the multiplexer module developed in b)
- d) Modify the testbench `updowncounter_tb.v` to include additional verifications required to validate this new function.

2.2 The second function (set the maximum count value) can be built around a circuit based on a magnitude comparator to activate the reset input of the register for forcing the counter return to zero. Note that the counter output should not exceed the maximum value set at input `max_count[ ]` and the counter will count with the step set at input `step_count[ ]`. For example, considering the counter is counting up, with `step_count` set to 3 and `max_count` set to 13, the sequence of values generated should be:

0, 3, 6, 9, 12, 0, 3, ...

When the counter is counting down and the next counter value exceeds the maximum, the counter should be loaded with that maximum value. For example, considering the counter is counting down, with `step_count` set to 3 and `max_count` set to 13, the sequence of values generated should be:

0, 13, 10, 7, 4, 1, 13

- a) draw a block diagram illustrating the modification required to the original circuit, based on a magnitude comparator and additional logic that might be necessary;
- b) build a behavioral module of a combinational 8-bit magnitude comparator, using the Verilog **assign** statement;
- c) include in your counter the comparator and any other additional logic necessary to implement the required functionality.
- d) adapt the testbench to verify this new functionality.